

# MEMORANDO TÉCNICO

## *Programação Paralela com MPI — Message Passing Interface*

---

**De:** Lúcio Nguendelamba Marcial Vitorino

**Nº de Matrícula:** 939032485

**Disciplina:** CPD

**Assunto:** Introduzir a programação com MPI (Message Passing Interface)

**Data:** 12 / 05 / 2026

### 1. Introdução

---

O presente trabalho insere-se no âmbito da unidade curricular de Computação Paralela e Distribuída (CPD) e tem como objetivo introduzir os conceitos fundamentais da programação paralela através da tecnologia MPI (Message Passing Interface). O MPI é uma norma de comunicação amplamente utilizada em sistemas de computação de alto desempenho, permitindo que múltiplos processos executem concorrentemente e troquem informação de forma estruturada e eficiente.

O desenvolvimento deste projeto baseia-se na análise, modificação e execução de programas que implementam diferentes paradigmas de comunicação entre processos. Em particular, são exploradas topologias em anel, nas quais os processos enviam e recebem mensagens de forma sequencial e circular. A partir dessas implementações, realizam-se medições experimentais de desempenho focadas na latência de comunicação e na largura de banda da rede.

Para além das medições de desempenho, o projeto propõe uma comparação entre estratégias distintas de comunicação — nomeadamente o envio ponto-a-ponto e o uso de operações coletivas como o broadcast — com o objetivo de avaliar o impacto de cada abordagem no comportamento global do sistema. O trabalho visa assim consolidar competências teóricas e práticas em computação paralela, com especial enfoque na eficiência de comunicação em ambientes distribuídos.

## 2. Experiências Realizadas

### a) Análise do Código — sendReceive

O programa `sendReceive` implementa um padrão de comunicação em anel utilizando primitivas MPI de envio e recepção ponto-a-ponto. Nesta topologia, cada processo envia uma mensagem ao processo imediatamente seguinte (com rank  $id + 1$ ), enquanto o processo com o rank mais elevado retorna a mensagem ao processo raiz (rank 0), fechando assim o ciclo. O número de vezes que a mensagem percorre o anel completo é controlado pela variável `rounds`.

O principal objetivo do programa é medir o custo temporal das operações de envio (`MPI_Send`) e recepção (`MPI_Recv`), permitindo caracterizar o desempenho da comunicação inter-processos. A medição do tempo é efetuada com recurso à função `MPI_Wtime()`, que fornece uma resolução temporal de alta precisão.

### b) Compilação e Execução

O programa é compilado com o wrapper `mpicc`, que integra automaticamente as bibliotecas MPI necessárias. A compilação é realizada através do seguinte comando:

```
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpicc sendReceive.c
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 3
Rounds= 3, N Processes = 5, Time = 0.000114 sec,
Average time per Send/Recv = 3.80 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 2
Rounds= 2, N Processes = 5, Time = 0.000098 sec,
Average time per Send/Recv = 4.88 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 3
Rounds= 3, N Processes = 5, Time = 0.000114 sec,
Average time per Send/Recv = 3.80 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 5
Rounds= 5, N Processes = 5, Time = 0.000111 sec,
Average time per Send/Recv = 2.22 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 8
Rounds= 8, N Processes = 5, Time = 0.000122 sec,
Average time per Send/Recv = 1.52 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$ mpirun -np 5 ./a.out 20
Rounds= 20, N Processes = 5, Time = 0.000227 sec,
Average time per Send/Recv = 1.13 us
• lucio-vitorino@lucio-vitorino-IdeaPad-3-15IAU7:~/Documentos/CPD/lab4$
```

### c) Interpretação dos Resultados

Os resultados experimentais obtidos apresentam as seguintes métricas de desempenho:

**Rounds:** número de vezes que a mensagem percorreu o anel completo de comunicação entre todos os processos participantes.

**N Processes:** número total de processos MPI envolvidos na execução, definido pelo parâmetro `-np` do `mpirun`. Neste conjunto de experiências, foi utilizado consistentemente o valor 5.

**Time:** tempo total, em segundos, necessário para a conclusão de todas as operações MPI\_Send e MPI\_Recv ao longo das N rondas, medido pelo processo raiz (rank 0).

**Average time per Send/Recv:** tempo médio por operação elementar de comunicação, calculado dividindo o tempo total pelo número total de operações realizadas ( $2 \times \text{rounds} \times N$  processos). Expresso em microssegundos ( $\mu\text{s}$ ).

Tabela de Resultados Experimentais

| Rounds | Nº Processos | Tempo Total (s) | Tempo Médio por Send/Recv ( $\mu\text{s}$ ) |
|--------|--------------|-----------------|---------------------------------------------|
| 2      | 5            | 0.000098        | 4.88                                        |
| 3      | 5            | 0.000114        | 3.80                                        |
| 5      | 5            | 0.000111        | 2.22                                        |
| 8      | 5            | 0.000122        | 1.52                                        |
| 20     | 5            | 0.000227        | 1.13                                        |

A análise dos resultados revela uma tendência clara: à medida que o número de rondas aumenta, o tempo médio por operação Send/Recv diminui progressivamente. Este comportamento deve-se ao efeito de amortização — com mais iterações, os custos fixos de inicialização e sincronização MPI são distribuídos por um maior número de operações, reduzindo o custo médio por mensagem. Com 20 rondas, o tempo médio atingiu 1,13  $\mu\text{s}$ , o valor mais reduzido do conjunto, evidenciando uma maior estabilidade da medição com um número elevado de amostras.

### 3. Desafios — Medição de Latência e Largura de Banda

Com o objetivo de caracterizar de forma mais completa o subsistema de comunicação MPI, foi desenvolvido um programa estendido capaz de medir simultaneamente a latência de comunicação e a largura de banda da rede. Este programa aceita dois parâmetros: o número de rondas e o tamanho da mensagem em bytes, permitindo uma análise paramétrica do desempenho em função do volume de dados transmitido.

#### 3.1 Implementação

O código apresentado em seguida implementa a topologia em anel já descrita, generalizando-a para mensagens de tamanho variável e incorporando o cálculo das métricas de latência e largura de banda:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int id, p, rounds, size;
    double start, end, elapsed;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    if (argc != 3) {
        if (id == 0)
            printf("Uso: %s <rounds> <message_size_bytes>\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    rounds = atoi(argv[1]);
    size = atoi(argv[2]);

    char *buffer = (char *)malloc(size);
    if (buffer == NULL) {
        fprintf(stderr, "Erro: falha na alocação de memória\n");
        MPI_Finalize();
        return 1;
    }

    MPI_Barrier(MPI_COMM_WORLD); /* sincronização antes da medição */
    start = MPI_Wtime();

    for (int i = 0; i < rounds; i++) {
        if (id == 0) {
            MPI_Send(buffer, size, MPI_CHAR, 1, i, MPI_COMM_WORLD);
            MPI_Recv(buffer, size, MPI_CHAR, p-1, i, MPI_COMM_WORLD,
                     MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(buffer, size, MPI_CHAR, id-1, i, MPI_COMM_WORLD,
                     MPI_STATUS_IGNORE);
            MPI_Send(buffer, size, MPI_CHAR, (id+1) % p, i, MPI_COMM_WORLD);
        }
    }

    end = MPI_Wtime();
    elapsed = end - start;

    if (id == 0) {
        double latency = (elapsed * 1e6) / (2.0 * rounds * p);
        double bandwidth = ((double)size * rounds * p) / elapsed;
        printf("Rounds = %d, Processos = %d\n", rounds, p);
        printf("Tamanho da mensagem = %d bytes\n", size);
        printf("Tempo total = %.6lf s\n", elapsed);
    }
}
```

```
    printf("Latência por Send/Recv = %.3lf µs\n", latency);  
    printf("Largura de banda      = %.3lf MB/s\n",  
           bandwidth / (1024.0 * 1024.0));  
}  
  
free(buffer);  
MPI_Finalize();  
return 0;  
}
```

## 3.2 Análise das Métricas

O programa calcula duas métricas fundamentais para a caracterização de um canal de comunicação em sistemas distribuídos:

**Latência (Latency):** representa o tempo mínimo necessário para transmitir uma mensagem de tamanho mínimo entre dois processos, independentemente do volume de dados. É calculada como o quociente entre o tempo total e o dobro do produto do número de rondas pelo número de processos ( $2 \times \text{rounds} \times p$ ), uma vez que cada ronda envolve um par Send/Recv por processo. Exprime-se em microssegundos ( $\mu\text{s}$ ).

**Largura de Banda (Bandwidth):** quantifica o volume de dados transferidos por unidade de tempo, medida em megabytes por segundo (MB/s). É calculada pelo quociente entre o volume total de dados transmitidos ( $\text{size} \times \text{rounds} \times p$ ) e o tempo total decorrido.

A separação entre latência e largura de banda é essencial para diagnósticos de desempenho: a latência domina o custo de mensagens pequenas e frequentes, enquanto a largura de banda determina a eficiência na transferência de grandes blocos de dados. A utilização de MPI\_Barrier antes do início da medição garante que todos os processos comecem a comunicar de forma sincronizada, eliminando desvios temporais devidos a arranques assíncronos.

## 4. Conclusão

---

O presente trabalho permitiu adquirir e consolidar competências fundamentais na programação paralela com MPI. Através da implementação e análise de uma topologia de comunicação em anel, foi possível observar experimentalmente como os custos de comunicação inter-processos evoluem em função do número de rondas e do tamanho das mensagens transmitidas.

Os resultados obtidos demonstram que o aumento do número de rondas contribui para uma estimativa mais estável e precisa do tempo médio por operação de comunicação, dado que os custos fixos de inicialização e sincronização são diluídos por um maior número de

iterrações. Esta observação é consistente com a teoria subjacente ao modelo de desempenho MPI, que distingue entre latência — dominante para mensagens curtas — e largura de banda — relevante para grandes transferências.

A extensão do programa para medir simultaneamente latência e largura de banda representa um passo relevante na direção da avaliação quantitativa de sistemas distribuídos. As metodologias e ferramentas exploradas neste trabalho constituem a base para a análise de algoritmos paralelos mais complexos, nomeadamente aqueles que envolvem comunicações coletivas como MPI\_Bcast, MPI\_Reduce ou MPI\_Allreduce, a explorar em trabalhos futuros.

## 5. Referências Bibliográficas

---

- [1] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. MPI Forum, 2023. Disponível em: <https://www.mpi-forum.org/docs/>
- [2] Pacheco, P. S. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.
- [3] Gropp, W., Lusk, E., & Skjellum, A. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 3.<sup>a</sup> ed., 2014.

## 6. Repositório GitHub

---

<https://github.com/LucioVitorino/Lab4.git>